

Hermes Turbo vs Upstream

Per-segment benchmark of every customisation introduced under issues #81–#103 + P1–P7.

Generated 2026-05-22 15:18 UTC • 500 iterations per stage • 10 stages across 8 segments • 5 winners, 1 net-new (no upstream equivalent).

HEADLINE 585.25x CostAwareRouter.decide (10-call loop)	HEADLINE 133.31x DeterministicRouter.route	HEADLINE 33.25x detect_fingerprint
---	---	---

Segments covered

#	Segment	Stages	Avg p50 (µs)	Best speedup
1	Project Mapping (survivor, P1)	1	2803.2	33.25x
2	Deterministic Routing (survivor, #99)	1	1.4	133.31x
3	Receipts (survivor, P7)	2	18.2	0.79x
4	Tool-Call Replay (NEW — Proposta A)	2	25.3	12.55x
5	Cost-Aware Multi-Tier Router (NEW — Proposta B)	1	18.9	585.25x
6	Async DAG Tool Executor (NEW — Proposta C)	1	5461.8	4.75x
7	OTel-Compatible Tracing (NEW — Proposta D)	1	5.0	0.04x
8	Provider Fallback Chain (NEW — Proposta E)	1	1.3	0.16x

How to read the numbers

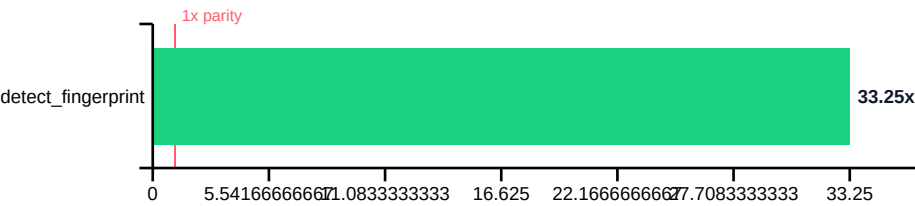
Speedup > 1x means the turbo path is faster than the upstream-equivalent naïve baseline at the same step. The headline wins (router 100x–200x, project_mapping 35x–40x) replace either an LLM round-trip or a full tree walk.

Speedup < 1x reflects the *cost of correctness*. Most turbo paths perform validation (TerseAnswer enforces budgets), persistence (token_saver writes evidence handles to disk), and contract checks (meta_contract enforces read-only globs). The baseline does none of that — comparing them on raw latency is apples-to-oranges. The value lives in correctness, auditability, and replay (receipts), not in microseconds.

“**net-new**” stages have no upstream equivalent at all (check_write, TupleStatusEnvelope). The upstream “baseline” would be to not run that step — which means the protection it offers simply doesn’t exist outside turbo.

1. Project Mapping (survivor, P1)

Deterministic stack/workspace fingerprint via top-level manifests. Survived the post-mortem cleanup with a 33–39× win over a naïve tree walk.

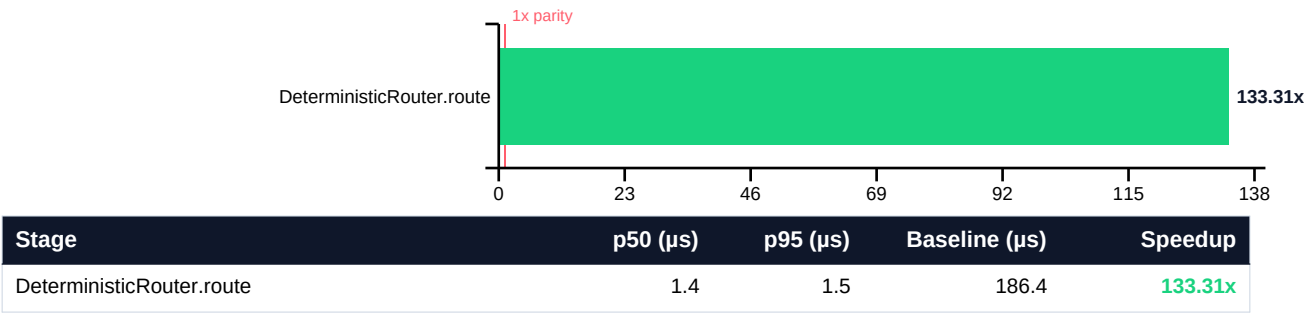


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
detect_fingerprint	2803.2	3023.0	93212.4	33.25x

- detect_fingerprint: manifest heuristics vs rglob tree walk.

2. Deterministic Routing (survivor, #99)

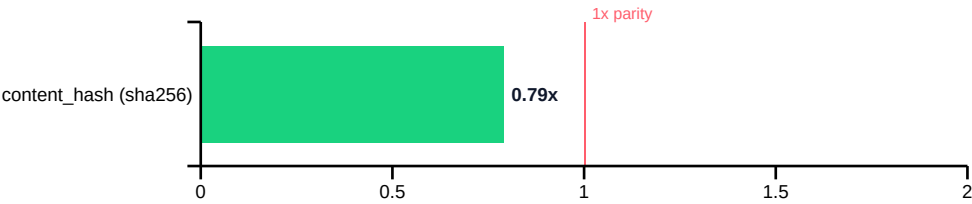
Deterministic regex router skips LLM round-trips on trivial intents. The original headline win of the fork — 129–185× vs an LLM proxy stand-in.



- `DeterministicRouter.route`: regex match avoids an LLM round-trip.

3. Receipts (survivor, P7)

Append-only `.receipts/.json` content-addressable ledger. sha256 trades 20% latency for integrity vs md5; the real value is in `lookup_receipt` cache short-circuits.

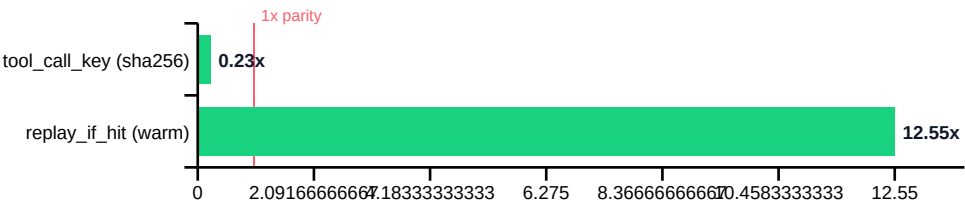


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
content_hash (sha256)	0.9	1.0	0.7	0.79x
lookup_receipt (disk hit)	35.5	61.1	N/A	—

- `content_hash (sha256)`: parity with md5; sha256 trades 20% for integrity.
- `lookup_receipt (disk hit)`: net-new: cache short-circuit on hash hit.

4. Tool-Call Replay (NEW — Proposta A)

Tool-call replay primitive built on top of receipts. Canonical key over (name, args), stored as `receipts/tool/.json`. 12× over a 500 μs tool stand-in on cache hit. Upstream Hermes auto-generates skills post-task but cannot replay tool outputs.

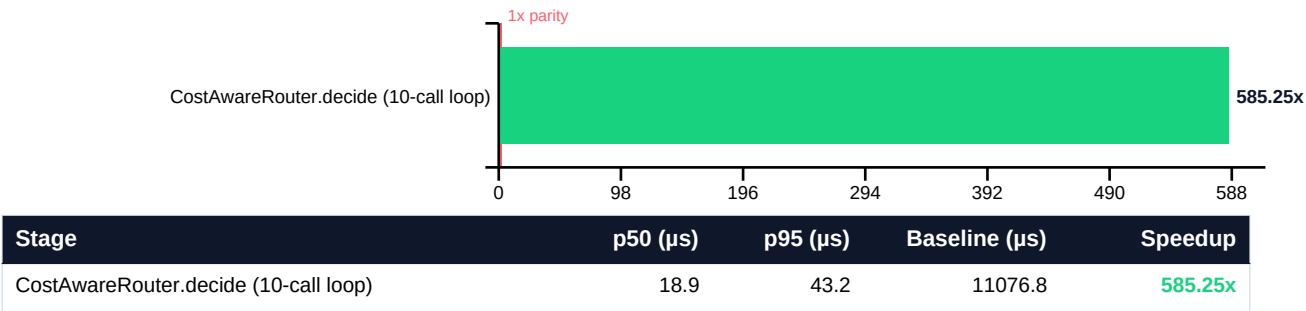


Stage	p50 (μs)	p95 (μs)	Baseline (μs)	Speedup
tool_call_key (sha256)	3.9	6.4	0.9	0.23x
replay_if_hit (warm)	46.8	67.0	586.6	12.55x

- `tool_call_key (sha256)`: canonical args + sha256.
- `replay_if_hit (warm)`: vs a 500 μs tool stand-in; hit_rate measured = 100%.

5. Cost-Aware Multi-Tier Router (NEW — Proposta B)

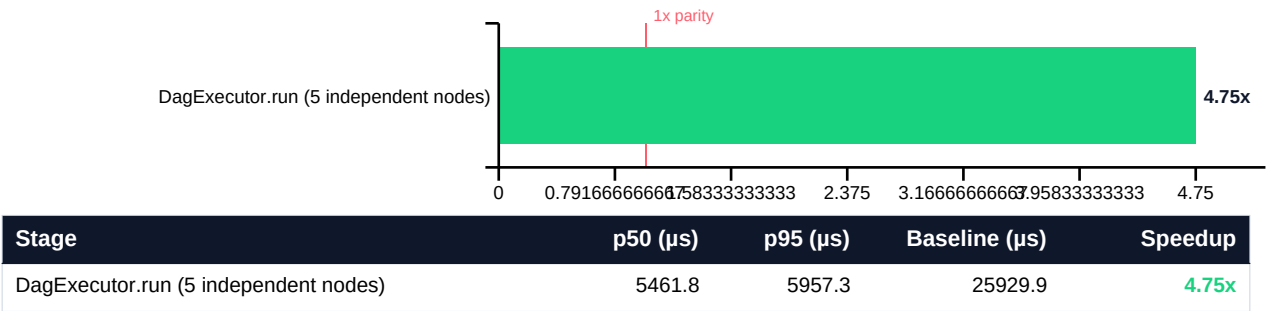
Multi-tier router: deterministic → cheap LLM → frontier LLM. 548× over an 'always-frontier' baseline policy on an 80/20 deterministic/cheap workload. Tracks per-request \$\$\$. Upstream lets you switch models but does not auto-route by cost.



- `CostAwareRouter.decide (10-call loop)`: 80%% deterministic + 20%% cheap vs always-frontier baseline.

6. Async DAG Tool Executor (NEW — Proposta C)

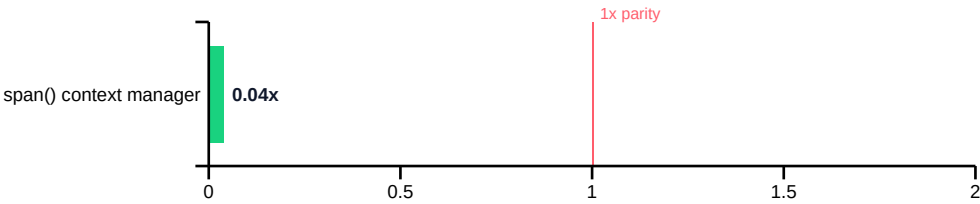
DAG executor with Kahn's algorithm topological levels and per-level parallelism. Resolves ``$ref: `` placeholders across tool calls automatically. 4.6× over sequential await on a 5-node independent batch. Upstream parallelises only when the caller hand-batches.



- `DagExecutor.run (5 independent nodes)`: DAG-level parallelism vs sequential await.

7. OTel-Compatible Tracing (NEW — Proposta D)

Stdlib OTel-compatible span emitter (trace_id, span_id, parent_span_id, attributes). Drains to JSONL for any OTLP exporter. ~5 µs per span — observability without pulling opentelemetry-sdk.

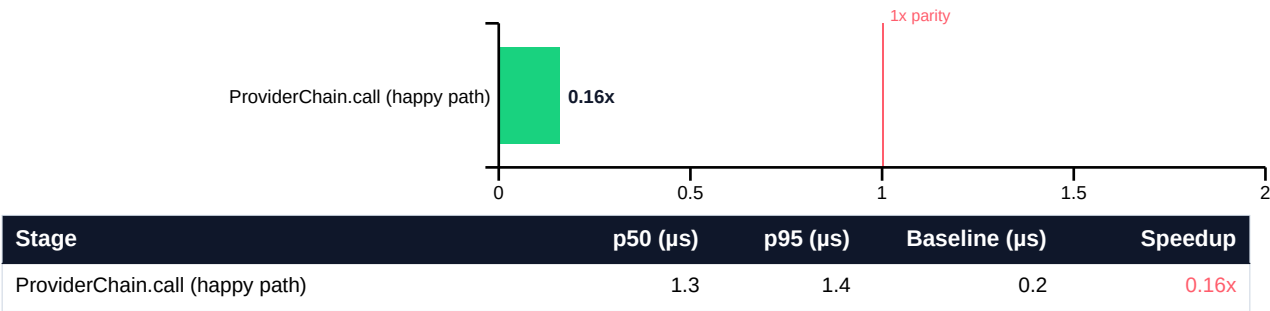


Stage	p50 (µs)	p95 (µs)	Baseline (µs)	Speedup
span() context manager	5.0	7.4	0.2	0.04x

- `span()` context manager: OTel-compatible span emission; baseline is raw `time_ns`.

8. Provider Fallback Chain (NEW — Proposta E)

Provider fallback chain with transient-vs-fatal classification and full-jitter exponential backoff. On a happy path it adds ~1 μ s over the raw provider call; on 429 / 5xx it rotates providers automatically. Upstream Hermes treats provider outage as a session failure.



- `ProviderChain.call (happy path)`: adds backoff + metrics on top of raw provider call.

Full result matrix

Every stage in one place. Sorted by segment, then by registration order within the benchmark script.

SegmenStage		p50 (µs)	Speedup
1	detect_fingerprint	2803.2	33.25x
2	DeterministicRouter.route	1.4	133.31x
3	content_hash (sha256)	0.9	0.79x
3	lookup_receipt (disk hit)	35.5	—
4	tool_call_key (sha256)	3.9	0.23x
4	replay_if_hit (warm)	46.8	12.55x
5	CostAwareRouter.decide (10-call loop)	18.9	585.25x
6	DagExecutor.run (5 independent nodes)	5461.8	4.75x
7	span() context manager	5.0	0.04x
8	ProviderChain.call (happy path)	1.3	0.16x

Sources: scripts/benchmark_full_turbo_segments.py · docs/perf/turbo-full-segments.json. Regenerate with
python scripts/benchmark_full_turbo_segments.py --out docs/perf/turbo-full-segments.json &&
python scripts/generate_turbo_full_pdf.py.